```
#include "HX711.h"

//initializing scales
HX711 scale1;
HX711 scale2;
HX711 scale3;
HX711 scale4;

int SCK1 = 11;
int DT1 = 10;

int SCK2 = 9;
int DT2 = 8;

int SCK3 = 7;
int DT3 = 6;

int SCK4 = 5;
int DT4 = 4;

//tare button
int tare_button = 3;
int last_state = HIGH;
bool do_tare = false;

//motor button
int reset_button = 2;
int last_reset = HIGH;
bool reset = 0;


//misc
int trials = 1;
unsigned long print_interval = 50;
unsigned long last_print = 0;
unsigned long curr_time = 0;

const int N = 20;

float buf1[N]={0};
```

```
float buf2[N]={0};
float buf3[N]={0};
float buf4[N]={0};
int avgInd = 0;

const int S = 10;
float xBuf[S] = {0};
float yBuf[S] = {0};
int stableInd = 0;
float stableThreshold = 0.5;

float getStd(float buf[], int n) {
  float mean = 0;
  for (int i = 0; i < n; i++) mean += buf[i];
  mean /= n;
  float variance = 0;
  for (int i = 0; i < n; i++) variance += (buf[i] - mean) * (buf[i] -
mean);
  return sqrt(variance / n);
}

void tareAll(HX711 &s1,HX711 &s2,HX711 &s3,HX711 &s4){
  delay(500);
  s1.tare();
  s2.tare();
  s3.tare();
  s4.tare();
}

void extremeTare(HX711 &scale1,HX711 &scale2,HX711 &scale3,HX711 &scale4){
  delay(500);
  tareAll(scale1,scale2,scale3,scale4);
  int iter = 50;
  int tareTrials = 1;
  int threshold = 100;
  int maxAttempts = 10;
  float scale1Avg=0;
  float scale2Avg=0;
  float scale3Avg=0;
  float scale4Avg=0;
```

```
  for(int i=0;i<maxAttempts;i++){
    scale1Avg=0;
    scale2Avg=0;
    scale3Avg=0;
    scale4Avg=0;
    tareAll(scale1,scale2,scale3,scale4);
    for(int i=0;i<iter;i++){
      scale1Avg += scale1.get_units(tareTrials);
      scale2Avg += scale2.get_units(tareTrials);
      scale3Avg += scale3.get_units(tareTrials);
      scale4Avg += scale4.get_units(tareTrials);
      delay(100);
    }
    scale1Avg /= iter;
    scale2Avg /= iter;
    scale3Avg /= iter;
    scale4Avg /= iter;
    if (abs(scale1Avg) <= threshold && abs(scale2Avg) <= threshold &&
abs(scale3Avg) <= threshold && abs(scale4Avg) <= threshold){break;}

  }
  for(int i = 0; i < N; i++){
    buf1[i] = scale1.get_units(1);
    buf2[i] = scale2.get_units(1);
    buf3[i] = scale3.get_units(1);
    buf4[i] = scale4.get_units(1);
    delay(100);
  }
  avgInd = 0;
}


float getAvg(float buf[], int N){
  float sum = 0;
  for(int i=0;i<N;i++){
    sum += buf[i];
  }
  return sum / N;
}
```

```
float C[3][4] = {
    {-0.000023, -0.000023, -0.000026, -0.000023},
    {0.002427, -0.002570, -0.003096, 0.002549},
    {-0.002191, -0.002156, 0.002272, 0.002090},
};

double offsetX = 29.9; //in mm, THESE ARE REAL VALUES
double offsetY = 40.15; //in mm, THESE ARE REAL VALUES

int plateSide = 200; //200mm x 200mm plate

const double pi = 3.1415926;

double posX = 0; // FROM LOAD CELL
double posY = 0; // FROM LOAD CELL

double rX = posX + offsetX + (plateSide/2.0);
double rY = posY + offsetY + (plateSide/2.0);


double L1 = 169.85; //REAL
double L2 = 24.5; //REAL
double L3 = 170; //REAL
double d = sqrt((L1*L1)+(L2*L2));

int A2Pins[4]={37,34,36,35};

int stepPin = 26;
int dirPin = 27;
int enPin = 24;

int sequence[8][4] = {
  {1,0,0,0},
  {1,1,0,0},
  {0,1,0,0},
  {0,1,1,0},
  {0,0,1,0},
  {0,0,1,1},
```

```
  {0,0,0,1},
  {1,0,0,1}
};

float A1CurrAngle = 180;
float A2CurrAngle = 0;

void moveStepper2(int pins[], float &currAngle, float targetAngle, float
modifier, int speed){

  //float angle = currAngle;

  int direction;
  if (targetAngle >= currAngle) direction = 1;
  else direction = -1;

  float travel = abs(currAngle - targetAngle);
  int numSteps = round(travel/modifier);
  int count = 0;

  while (count< numSteps){
    for (int i=0;i<8;i++){
      if (count>=numSteps){break;}
      int step = i;
      if (direction == -1){
        step = 7-i;
        }

      for (int j=0;j<4;j++){
        digitalWrite(pins[j],sequence[step][j]);
      }

      currAngle += modifier*direction;

      delayMicroseconds(speed);
      count++;

    }
  }
```

```
}

void moveStepper1(float &currAngle, float targetAngle, int speed){
  if (currAngle == targetAngle){return;}

  digitalWrite(enPin,LOW);

  float stepPerDeg = 1600.0/360.0;

  if (targetAngle > currAngle){digitalWrite(dirPin,HIGH);}
  else {digitalWrite(dirPin,LOW);}

  float travel = abs(targetAngle-currAngle);
  int numSteps = round(travel*stepPerDeg);
  int direction = 1;
  if (targetAngle < currAngle){direction = -1;}

  for (int i=0; i<numSteps;i++){
    digitalWrite(stepPin,HIGH);
    delayMicroseconds(speed);
    digitalWrite(stepPin,LOW);
    delayMicroseconds(speed);
    currAngle += (direction/stepPerDeg);
  }

}

  float A1Mod = 0.9;
  float A2Mod = 0.3515625;
  int ready = 1;
  int moved = 0;

  float x_com = 0;
  float y_com = 0;

void setup() {
  // put your setup code here, to run once:
  for (int i=0;i<4;i++){
    pinMode(A2Pins[i],OUTPUT);
  }
```

```
  pinMode(stepPin, OUTPUT);
  pinMode(dirPin, OUTPUT);
  pinMode(enPin, OUTPUT);

  digitalWrite(enPin, LOW);
  digitalWrite(dirPin,HIGH);

  Serial.begin(115200);
  scale1.begin(DT1,SCK1);
  scale2.begin(DT2,SCK2);
  scale3.begin(DT3,SCK3);
  scale4.begin(DT4,SCK4);

  scale1.set_scale(1.0f);
  scale2.set_scale(1.0f);
  scale3.set_scale(1.0f);
  scale4.set_scale(1.0f);

  pinMode(tare_button,INPUT_PULLUP);
  pinMode(reset_button,INPUT_PULLUP);
  delay(500);
  scale1.tare();
  scale2.tare();
  scale3.tare();
  scale4.tare();
  delay(2000);
  extremeTare(scale1,scale2,scale3,scale4);
  //Serial.println("NOW");
  delay(3000);
}

void loop() {
  // put your main code here, to run repeatedly:
  double values[4];
  float outputs[3] = {0, 0, 0};

  if (scale1.is_ready() && scale2.is_ready() && scale3.is_ready() &&
scale4.is_ready()) {
    buf1[avgInd] = scale1.get_units(trials);
    buf2[avgInd] = scale2.get_units(trials);
```

```
  buf3[avgInd] = scale3.get_units(trials);
  buf4[avgInd] = scale4.get_units(trials);

  avgInd = (avgInd + 1) % N;
  values[0] = getAvg(buf1, N);
  values[1] = getAvg(buf2, N);
  values[2] = getAvg(buf3, N);
  values[3] = getAvg(buf4, N);

  outputs[0] = 0; outputs[1] = 0; outputs[2]=0;
  for(int i = 0; i < 3; i++){
    for(int j = 0; j < 4; j++){
        outputs[i] += C[i][j] * values[j];
    }
  }
  float W_meas = outputs[0];

  if (W_meas > 0.3){
    x_com  = outputs[1] / W_meas;
    y_com  = outputs[2] / W_meas;
    Serial.print(x_com); Serial.print(",");
    Serial.print(y_com); Serial.print(",");
    Serial.print(W_meas); Serial.print(",");
    Serial.println(moved);


    xBuf[stableInd] = x_com;
    yBuf[stableInd] = y_com;
    stableInd = (stableInd + 1) % S;

  }
  else{
    //x_com = 0.0;
    //y_com = 0.0;
    Serial.println("TOO LIGHT");
  }
  //Serial.print(x_com); Serial.print(","); Serial.println(y_com);
}

if (digitalRead(reset_button) == LOW && last_reset == HIGH){
```

```
    ready = 1;
    A1CurrAngle = 180;
    A2CurrAngle = 0;
    moved = 0;
  }
  last_reset = digitalRead(reset_button);
  //Serial.println(move_motor);

  xBuf[stableInd] = x_com;
  yBuf[stableInd] = y_com;

  float xStd = getStd(xBuf,S);
  float yStd = getStd(yBuf,S);

  //if(move_motor == true){
  if (xStd < stableThreshold && yStd < stableThreshold && x_com != 0.0 &&
y_com != 0.0 && ready == 1){
    ready = 0;
    double rX = x_com + offsetX + (plateSide/2.0);
    double rY = y_com + offsetY + (plateSide/2.0);

    double t2 = acos(((rX*rX)+(rY*rY)-(d*d)-(L3*L3))/(2.0*d*L3));
    double t1 = atan2(rY,rX)-atan2(L3*sin(t2),d+(L3*cos(t2)));
    t1 = t1 - atan(L2/L1) + pi/2;
    t2 += atan(L2/L1);
    double t2Motor = t2 * 180/pi;
    double t1Motor = t1 * 180/pi;
    delay(2000);
    moveStepper1(A1CurrAngle, t1Motor, 5000);
    delay(500);
    moveStepper2(A2Pins, A2CurrAngle, t2Motor, A2Mod, 5000);
    ready = 0;
    moved = 1;
  }

  if (digitalRead(tare_button) == LOW && last_state == HIGH){
    do_tare = true;
  }
  last_state = digitalRead(tare_button);
  if (do_tare == true){
```

```
    delay(1500);
    extremeTare(scale1,scale2,scale3,scale4);
    do_tare = false;
    //Serial.println("TARED!!");
    delay(500);
  }

}
```